

HireIQ

AI Hiring Intelligence Platform

Technical Architecture Document (TAD)

Document Type	Technical Architecture Document (TAD)
Product	HireIQ — AI Hiring Intelligence Platform
Version	v1.0.0
Parent Document	HireIQ PRD v1.0
Status	DRAFT — Engineering Review
Author	Engineering Team
Date	June 13, 2026
Classification	Internal — Engineering Confidential

SECTION 1

Architecture Overview & Guiding Principles

1. Architecture Overview & Guiding Principles

This document defines the complete technical architecture for HireIQ. It covers system design, service boundaries, data flows, infrastructure topology, API contracts, security architecture, ML system design, and deployment strategy. All engineering decisions are traceable to the HireIQ PRD v1.0.

1.1 Architecture Style

HireIQ is built using a Service-Oriented Architecture (SOA) with three independently deployable services:

Core API Service	The main backend handling all business logic: users, jobs, applications, resumes. Built with FastAPI (Python). Connected to PostgreSQL and Redis.
AI / ML Service	A dedicated microservice for all AI inference: resume parsing, candidate scoring, job recommendations, interview evaluation. Independently scalable.
Frontend Application	A Next.js (React) application consuming both services via a unified API gateway. Served via CloudFront CDN for global performance.

1.2 Guiding Architecture Principles

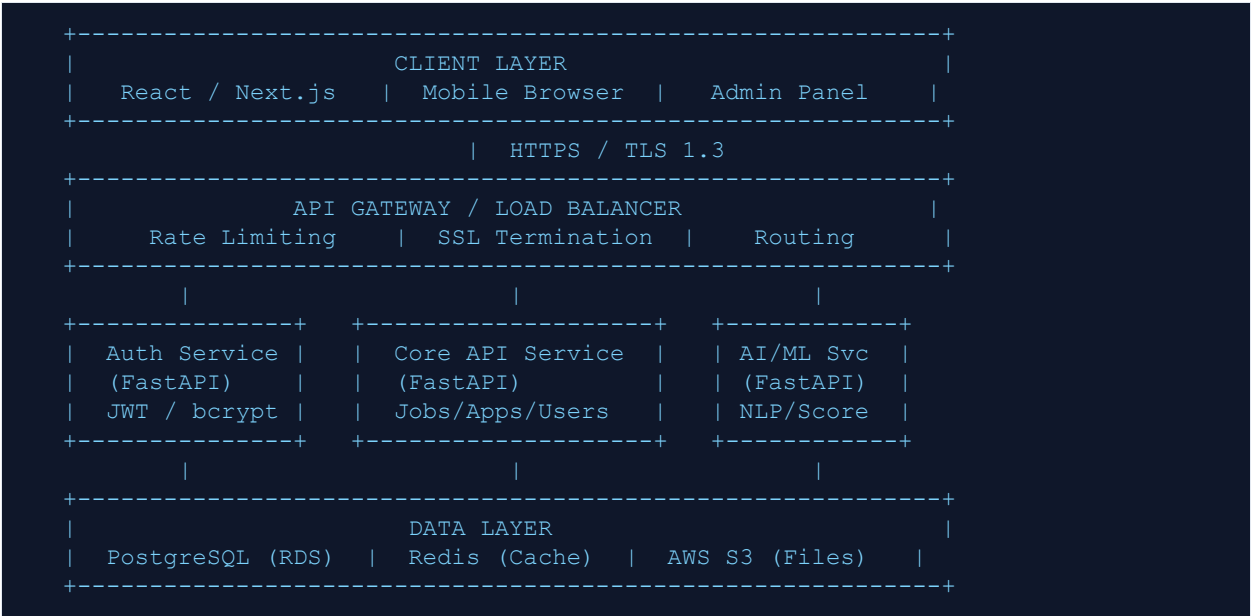
Separation of Concerns	Each service owns its domain. The AI service has no direct DB access — it receives data via Core API and returns inference results only.
Stateless Services	All services are stateless. Session state lives in JWTs (client-side) and Redis (server cache). Zero in-memory session storage.
Async Where Possible	AI inference runs asynchronously post-application-submit. User gets immediate HTTP 201; score appears when inference completes.
API-First Design	All business logic is exposed as REST endpoints before frontend is built. Frontend consumes APIs — zero direct DB access from the client.
Defence in Depth	Security applied in layers: API Gateway (rate limiting, WAF) → Service layer (JWT, RBAC) → Data layer (encryption at rest, pre-signed URLs).
Observable by Default	Structured logging, health checks, and metrics endpoints are built into every service from day one — not retrofitted later.
Infrastructure as Code	All AWS infrastructure is provisioned via Terraform. Zero manual console configuration in staging or production environments.

SECTION 2

System Architecture Diagram

2. System Architecture Diagram

2.1 High-Level System Architecture



2.2 Request Flow — Job Application with AI Scoring

Step-by-step data flow when a job seeker submits an application, triggering automatic AI scoring:

Step 1	User clicks Apply on the React frontend. An HTTP POST is sent to the API Gateway with the JWT Bearer token.
Step 2	API Gateway validates the JWT and routes the request to the Core API Service on the private network.
Step 3	Core API validates: job is active, user has a resume uploaded, no duplicate application exists. Creates application record with status='applied'.
Step 4	Core API makes an internal async HTTP call to the AI Service: POST /ai/score-candidate with resume text and job description.
Step 5	AI Service vectorizes both inputs via TF-IDF, computes cosine similarity, identifies skill gaps, and returns the scoring JSON payload.

Step 6	Core API updates the application record with ai_score, matched_skills, and missing_skills in PostgreSQL. Caches result in Redis.
Step 7	Core API returns HTTP 201 Created to the frontend with application_id and initial status. UI shows 'Application Submitted'.
Step 8	On dashboard load, score is served from Redis cache (fast path, ~2ms) or PostgreSQL fallback (cache miss, ~15ms).

2.3 Component Communication Map

From	To	Communication Method
Frontend	Core API	HTTPS REST via Axios. JWT Bearer token in Authorization header on every request.
Frontend	AI Service	Never directly. All AI calls are proxied through Core API — AI service is not publicly exposed.
Core API	PostgreSQL	Direct TCP via SQLAlchemy async ORM. Connection pool: min 5, max 20 connections.
Core API	Redis	redis-py client. Used for caching results and rate limit counters. Default TTL: 300s.
Core API	AWS S3	boto3 SDK. Generates pre-signed upload URLs (15 min expiry). Stores resulting file URL in DB.
Core API	AI Service	Internal HTTP REST on private VPC network. FastAPI to FastAPI. Not exposed to public internet.
AI Service	PostgreSQL	No direct access. All data passed via Core API request payload.
AI Service	Redis	Caches fitted TF-IDF vectorizer and job matrix to avoid reload on every inference call.

SECTION 3

Technology Stack

3. Technology Stack

3.1 Backend — Core API Service

Language	Python 3.11+
Framework	FastAPI — chosen for automatic OpenAPI/Swagger docs, native async support, and Pydantic validation
ORM	SQLAlchemy 2.0 with Alembic for database migrations
Auth	python-jose for JWT encode/decode; passlib[bcrypt] for password hashing (cost factor 12)
DB Driver	asyncpg — async PostgreSQL driver for high-concurrency non-blocking DB access
File Uploads	boto3 (AWS SDK) — generates S3 pre-signed upload URLs; no binary data passes through API
Caching	redis-py with hiredis parser for maximum Redis throughput
Validation	Pydantic v2 models for all request and response schemas — strict mode enabled
PDF Parsing	PyMuPDF (fitz) for text extraction from uploaded PDF resumes
Testing	pytest + httpx for API integration tests; pytest-asyncio for async test runner
Logging	structlog — structured JSON logging; shipped to AWS CloudWatch Logs
Background Tasks	FastAPI BackgroundTasks (v1); Celery + Redis queue for heavy workloads (v1.1)

3.2 AI / ML Service

Language	Python 3.11+
Framework	FastAPI — same ecosystem as Core API; lightweight for ML inference serving
NLP Engine	spaCy with en_core_web_lg model for Named Entity Recognition and tokenization
Skill Extraction	Custom 2,000+ skill dictionary + spaCy NER pipeline for technology skill identification

Vectorization	scikit-learn TfidfVectorizer — pre-fitted on 10,000+ job description corpus
Similarity	sklearn.metrics.pairwise.cosine_similarity — resume vector vs job description vector
Interview Eval	Sentence-Transformers (all-MiniLM-L6-v2) for semantic similarity of candidate answers
Model Storage	Models serialized with joblib; loaded at service startup to eliminate cold-start latency

3.3 Frontend

Framework	Next.js 14 (App Router) — SSR, SEO, and file-based routing
Language	TypeScript — strict mode on all components and utilities
Styling	Tailwind CSS v3 — utility-first with custom HireIQ brand design tokens
State	Zustand for global auth/UI state; TanStack Query (React Query) for server/async state
HTTP Client	Axios — request interceptors inject JWT; response interceptors handle 401 auto-refresh
Charts	Recharts — React-native charting for all dashboard analytics
Components	shadcn/ui — accessible, headless components styled with Tailwind CSS
Forms	React Hook Form + Zod for form state and runtime schema validation
Notifications	react-hot-toast — lightweight toast system with 4s auto-dismiss
File Upload	react-dropzone for drag-and-drop PDF upload with real-time progress bar
Icons	Lucide React — consistent icon set across all pages
Testing	Jest + React Testing Library (unit); Playwright (E2E)

3.4 Infrastructure & DevOps

Cloud Provider	AWS — EC2, RDS, S3, ElastiCache, CloudWatch, Route 53, CloudFront, ECS Fargate
Containerization	Docker — each service has its own Dockerfile; docker-compose for local dev environment
Orchestration	AWS ECS (Fargate) — serverless containers in production; no EC2 management overhead in v1
Database	AWS RDS PostgreSQL 15 — Multi-AZ deployment for high availability in production
Cache	AWS ElastiCache Redis 7 — single-node in v1; cluster mode enabled in v1.5

File Storage	AWS S3 — private bucket with pre-signed URL access; server-side AES-256 encryption
CDN	AWS CloudFront — serves Next.js static assets and S3 resume files globally
DNS	AWS Route 53 — manages hireiq.io domain and api.hireiq.io subdomain
CI/CD	GitHub Actions — lint, test, build Docker, push to ECR, deploy to ECS on every merge to main
IaC	Terraform — all AWS resources defined as code in /infrastructure directory
Monitoring	AWS CloudWatch + Datadog — metrics, structured logs, dashboards, and alerting
SSL	AWS Certificate Manager — auto-renewing TLS 1.3 certificates on all endpoints

SECTION 4

Database Architecture

4. Database Architecture

4.1 Database Selection Rationale

PostgreSQL was selected as the primary database for the following technical reasons:

- Native JSONB columns — stores AI-parsed resume data and skill arrays without a separate NoSQL database
- Full-text search via GIN indexes — powers job keyword search without Elasticsearch in v1
- ACID compliance — critical for application submission integrity and duplicate prevention constraints
- Native array type support — `skills[]`, `required_skills[]`, `matched_skills[]` stored as first-class columns
- Excellent managed service support on AWS RDS with automatic backups, Multi-AZ, and read replicas

4.2 Full Schema Definition

Table: users

```
CREATE TABLE users (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  name        VARCHAR(255) NOT NULL,  
  email       VARCHAR(255) UNIQUE NOT NULL,  
  password_hash VARCHAR(255) NOT NULL,  
  role        VARCHAR(20) NOT NULL CHECK (role IN  
( 'job_seeker', 'recruiter', 'admin' )),  
  bio         TEXT,  
  skills      TEXT[] DEFAULT '{}',  
  experience  VARCHAR(100),  
  education   VARCHAR(255),  
  avatar_url  VARCHAR(500),  
  profile_pct INTEGER DEFAULT 0 CHECK (profile_pct BETWEEN 0 AND 100),  
  is_active   BOOLEAN DEFAULT TRUE,  
  created_at  TIMESTAMPTZ DEFAULT NOW(),  
  updated_at  TIMESTAMPTZ DEFAULT NOW()  
);  
CREATE INDEX idx_users_email ON users(email);  
CREATE INDEX idx_users_role  ON users(role);
```

Table: jobs

```
CREATE TABLE jobs (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  title       VARCHAR(255) NOT NULL,  
  description  TEXT NOT NULL,  
  required_skills TEXT[] DEFAULT '{}',  
  salary_min  INTEGER,  
  salary_max  INTEGER,  
  location    VARCHAR(255),  
  job_type    VARCHAR(50) CHECK (job_type IN  
( 'full_time', 'part_time', 'contract', 'remote' )),  
  status      VARCHAR(20) DEFAULT 'active'  
              CHECK (status IN ( 'active', 'closed', 'draft' )),  
  recruiter_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
  applicant_count INTEGER DEFAULT 0,  
  created_at  TIMESTAMPTZ DEFAULT NOW(),  
  updated_at  TIMESTAMPTZ DEFAULT NOW()  
);  
CREATE INDEX idx_jobs_recruiter ON jobs(recruiter_id);  
CREATE INDEX idx_jobs_status   ON jobs(status);  
CREATE INDEX idx_jobs_skills   ON jobs USING GIN(required_skills);  
CREATE INDEX idx_jobs_fts      ON jobs USING GIN(  
  to_tsvector('english', title || ' ' || description));
```

Table: resumes

```
CREATE TABLE resumes (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id     UUID UNIQUE NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
  file_url    VARCHAR(500) NOT NULL,  
  raw_text    TEXT,  
  parsed_skills TEXT[] DEFAULT '{}',  
  parsed_experience VARCHAR(100),  
  parsed_education VARCHAR(255),  
  parsed_projects TEXT[] DEFAULT '{}',  
  ai_score    INTEGER CHECK (ai_score BETWEEN 0 AND 100),  
  parse_status VARCHAR(20) DEFAULT 'pending',  
  created_at  TIMESTAMPTZ DEFAULT NOW(),  
  updated_at  TIMESTAMPTZ DEFAULT NOW()  
);  
CREATE INDEX idx_resumes_user ON resumes(user_id);
```

Table: applications

```
CREATE TABLE applications (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id     UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
  job_id      UUID NOT NULL REFERENCES jobs(id) ON DELETE CASCADE,  
  resume_url  VARCHAR(500) NOT NULL,  
  ai_score    INTEGER CHECK (ai_score BETWEEN 0 AND 100),  
  matched_skills TEXT[] DEFAULT '{}',  
  missing_skills TEXT[] DEFAULT '{}',
```

```

status          VARCHAR(20) DEFAULT 'applied'
                CHECK (status IN ('applied','shortlisted','rejected','hired')),
applied_at      TIMESTAMPTZ DEFAULT NOW(),
updated_at      TIMESTAMPTZ DEFAULT NOW(),
UNIQUE(user_id, job_id) -- enforces one-application-per-job at DB level
);
CREATE INDEX idx_apps_user    ON applications(user_id);
CREATE INDEX idx_apps_job     ON applications(job_id);
CREATE INDEX idx_apps_score   ON applications(job_id, ai_score DESC);
CREATE INDEX idx_apps_status  ON applications(status);

```

4.3 Indexing Strategy

Index Target	Type	Purpose
users.email	B-tree (unique)	Primary lookup for login — most frequent query in the auth flow
jobs.required_skills	GIN array index	Powers skill-based filtering: WHERE required_skills @> ARRAY['Python']
jobs.title + description	GIN full-text	Keyword search: WHERE to_tsvector(...) @@ plainto_tsquery('query')
applications(job_id, ai_score DESC)	Composite B-tree	Recruiter dashboard: applicants for a job ordered by AI score
applications(user_id, job_id)	Unique composite	DB-level enforcement of one-application-per-job constraint
resumes.user_id	B-tree (unique)	Fast one-to-one resume lookup for a given user

4.4 Redis Caching Strategy

Redis Key Pattern	TTL	Usage & Invalidation Trigger
job_list:{page}:{filters_hash}	300s	Paginated job listing. Invalidated on any job create/update/delete.
user_profile:{user_id}	600s	User profile data. Invalidated on profile update.
ai_score:{user_id}:{job_id}	3600s	AI scoring result. Invalidated on resume re-upload.
job_recs:{user_id}	1800s	Top 5 recommended jobs. Invalidated on profile skill update.
rate:{ip}:{endpoint}	60s	Auth rate limit counter. 100 requests/min per IP on /auth/*.
refresh:{user_id}	86400s	Refresh token store for JWT rotation flow.

SECTION 5

API Architecture & Endpoint Contracts

5. API Architecture & Endpoint Contracts

5.1 API Design Standards

Base URL	https://api.hireiq.io/v1 (production) http://localhost:8000/v1 (local development)
Protocol	HTTPS only in production (TLS 1.3). HTTP connections blocked at load balancer level.
Format	JSON request and response bodies. Content-Type: application/json required on all POST/PUT/PATCH.
Authentication	Bearer token: Authorization: Bearer <access_token> header on all protected routes.
Versioning	URL path versioning (/v1/, /v2/) — enables breaking changes without disrupting existing clients.
Pagination	Query params: ?page=1&limit=10. Response: { data: [], total, page, pages, has_next }.
Error Envelope	{ error: string, code: string, details?: object } — consistent across all error responses.
HTTP Status Codes	200 OK 201 Created 400 Bad Request 401 Unauthorized 403 Forbidden 404 Not Found 409 Conflict 422 Validation Error 500 Internal Error

5.2 Authentication Endpoints

Endpoint	Access	Purpose	Request Body	Response
POST /auth/register	Public	Register new user	{ name, email, password, role }	{ user_id, token, refresh_token }
POST /auth/login	Public	Authenticate user	{ email, password }	{ token, refresh_token, user }
POST /auth/refresh	Public	Refresh access token	{ refresh_token }	{ token, refresh_token }
POST /auth/logout	Auth	Invalidate refresh token	{ refresh_token }	{ message: string }

GET /auth/me	Auth	Get current user info	—	Full user object
-------------------------------	------	-----------------------	---	------------------

5.3 Job Endpoints

Endpoint	Access	Purpose	Request / Query Params	Response
GET /jobs	Auth	List jobs (paginated)	?page&limit&skills&location&salary_min&salary_max&search	Paginated job list
POST /jobs	Recruiter	Create job posting	{ title, description, required_skills[], salary_min, salary_max, location, job_type }	Created job object
GET /jobs/:id	Auth	Get job detail	—	Full job + recruiter info
PUT /jobs/:id	Recruiter (owner)	Update own job	Partial job fields	Updated job object
DELETE /jobs/:id	Recruiter (owner)	Delete own job	—	{ message }
PATCH /jobs/:id/close	Recruiter (owner)	Close job to new applications	—	{ status: closed }

5.4 Resume Endpoints

Endpoint	Access	Purpose	Request	Response
POST /resumes/upload	Job Seeker	Upload PDF to S3	multipart/form-data (PDF, max 5MB)	{ resume_id, file_url, parse_status }
GET /resumes/me	Job Seeker	Get own parsed resume	—	{ file_url, parsed_skills[], ai_score, ... }
DELETE /resumes/me	Job Seeker	Delete resume from S3 + DB	—	{ message }

5.5 Application Endpoints

Endpoint	Access	Purpose	Request	Response
POST /applications	Job Seeker	Apply to a job	{ job_id }	{ application_id, status, message }
GET /applications/me	Job Seeker	Own application history	?status&page&limit	Paginated application list

GET /applications/job/:job_id	Recruiter	All applicants for a job	?sort_by=ai_score&status&page	Ranked applicants
PATCH /applications/:id/status	Recruiter (job owner)	Update applicant status	{ status: shortlisted rejected hired }	Updated applicant object

5.6 AI Service Endpoints

Endpoint	Access	Purpose	Request Body	Response
POST /ai/analyze-resume	Any Auth	Parse resume text to structured JSON	{ resume_text: string }	{ skills[], experience, education, projects[] }
POST /ai/score-candidate	Any Auth	Score resume vs job description	{ resume_data: obj, job_description: string }	{ score: int, matched_skills[], missing_skills[] }
POST /ai/recommend-jobs	Job Seeker	Get top 5 personalized job recs	{ user_id: string }	{ recommendations: [{ job_id, title, score }] }
POST /ai/evaluate-interview	Any Auth	Score an interview answer	{ question: string, answer: string }	{ score, feedback, confidence }
GET /ai/health	Public	AI service health check	—	{ status, model_loaded, latency_ms }

5.7 Recruiter Dashboard Endpoints

Endpoint	Access	Purpose	Query Params	Response
GET /recruiter/jobs	Recruiter	All jobs by this recruiter	?status&page&limit	Paginated list with applications
GET /recruiter/analytics	Recruiter	Hiring analytics summary	?job_id (optional)	{ total, shortlisted, rejected, score_distribution[] }
GET /recruiter/top-candidates/:job_id	Recruiter	Top AI-scored candidates for a job	?limit=10	Top N candidates with scores and skills

SECTION 6

Folder Structure & Module Organization

6. Folder Structure & Module Organization

6.1 Backend — Core API

```
backend/
  app/
    api/v1/routes/
      auth.py           # Registration, login, logout, token refresh endpoints
      users.py          # Profile CRUD, avatar upload
      jobs.py           # Job CRUD, search, filter, pagination
      resumes.py        # PDF upload to S3, text extract, AI trigger
      applications.py   # Apply, status tracking, recruiter views
      ai.py             # Proxy to AI service; cache results in Redis
      recruiter.py      # Dashboard, analytics, top candidates endpoints
    core/
      config.py         # Pydantic BaseSettings — loads all env variables
      security.py       # JWT encode/decode, bcrypt hashing helpers
      database.py       # SQLAlchemy async engine and session factory
      redis.py          # Redis client singleton with connection pool
    models/             # SQLAlchemy ORM models
      user.py | job.py | resume.py | application.py | session.py
    schemas/            # Pydantic v2 request/response models
      user.py | job.py | resume.py | application.py | ai.py
    services/           # Business logic layer (controllers call services)
      user_service.py   # Profile completion, skill updates
      job_service.py    # Search, filter, pagination, status management
      resume_service.py # S3 upload, PDF extraction, parse trigger
      application_service.py # Apply logic, duplicate check, score fetch
      ai_service.py     # HTTP client to AI Service + Redis caching
      analytics_service.py # Aggregate queries for recruiter dashboard
    middlewares/
      auth_middleware.py # JWT verification on protected routes
      role_middleware.py # RBAC decorators: require_role('recruiter')
      rate_limit.py      # Redis-backed rate limiter middleware
      logging_middleware.py # Structured request/response JSON logging
    utils/
      s3.py             # boto3: upload, delete, generate pre-signed URLs
      pdf.py            # PyMuPDF text extraction utility function
      pagination.py     # Generic paginate() helper for SQLAlchemy queries
      validators.py     # Custom input validators (file type, size, etc.)
    migrations/         # Alembic migration scripts (auto-generated)
    tests/              # pytest test suites for all route modules
    main.py             # FastAPI app factory; register all routers
    requirements.txt    # Pinned Python dependencies
    Dockerfile          # Multi-stage build: deps + app
    .env.example        # Template for all required environment variables
```

6.2 AI / ML Service

```
ai_service/
  app/
    routes/
      analyze.py          # POST /ai/analyze-resume
      score.py            # POST /ai/score-candidate
      recommend.py        # POST /ai/recommend-jobs
      interview.py        # POST /ai/evaluate-interview
      health.py           # GET /ai/health
    engines/
      resume_engine.py    # spaCy NER pipeline + skill dict matcher
      scoring_engine.py   # TF-IDF vectorizer + cosine similarity scorer
      recommendation_engine.py # Content-based job recommendation logic
      interview_engine.py  # Sentence-transformer answer evaluator
    data/
      skills_dictionary.json # 2,000+ curated tech skills
      ideal_answers.json    # Interview ideal answer corpus
      job_corpus.pkl        # Pre-built TF-IDF job matrix (serialized)
    models/
      tfidf_vectorizer.pkl  # Fitted TF-IDF model (joblib serialized)
      sentence_transformer/ # Hugging Face model weights directory
    schemas/
      ai_schemas.py       # Pydantic request/response for all AI endpoints
    core/
      config.py            # AI service settings from environment
      model_loader.py      # Load all models at startup (eliminate cold start)
  tests/
    test_resume_engine.py
    test_scoring_engine.py
    test_interview_engine.py
  main.py
  requirements.txt
  Dockerfile
```

6.3 Frontend — Next.js

```
frontend/
  app/
    # Next.js 14 App Router
    (auth)/login/page.tsx
    (auth)/register/page.tsx
    (dashboard)/dashboard/page.tsx # Redirects by role
    (dashboard)/jobs/page.tsx      # Job listing + search
    (dashboard)/jobs/[id]/page.tsx # Job detail
    (dashboard)/jobs/create/page.tsx # Create job (recruiter)
    (dashboard)/resume/page.tsx
    (dashboard)/applications/page.tsx
    (dashboard)/ai/insights/page.tsx # AI score + skill gap
    (dashboard)/ai/interview/page.tsx # AI mock interview chat
    (dashboard)/recruiter/applicants/[jobId]/page.tsx
    (dashboard)/recruiter/analytics/page.tsx
```

```
(dashboard)/profile/page.tsx
admin/page.tsx
components/
  ui/                                # shadcn/ui base components
  layout/Navbar.tsx | Sidebar.tsx
  jobs/JobCard.tsx | JobFilter.tsx | JobSearch.tsx
  candidates/CandidateCard.tsx | CandidateRankList.tsx
  ai/AIScoreCard.tsx | SkillGapChart.tsx | InterviewChat.tsx
  charts/ApplicantTrendChart.tsx | SkillDistributionChart.tsx
  common/LoadingSkeleton.tsx | EmptyState.tsx | ProfileCompletionBar.tsx
  resume/ResumeUpload.tsx | ResumePreview.tsx
lib/
  api.ts                            # Axios instance with JWT interceptors
  auth.ts                           # Token helpers, role guards
  utils.ts                           # General utilities
hooks/
  useAuth.ts | useJobs.ts | useApplications.ts | useAI.ts
store/
  authStore.ts                      # Zustand: user, token, role
  uiStore.ts                         # Zustand: dark mode, sidebar collapse
types/index.ts                      # All TypeScript interfaces and types
tailwind.config.ts | next.config.ts | tsconfig.json
Dockerfile
```

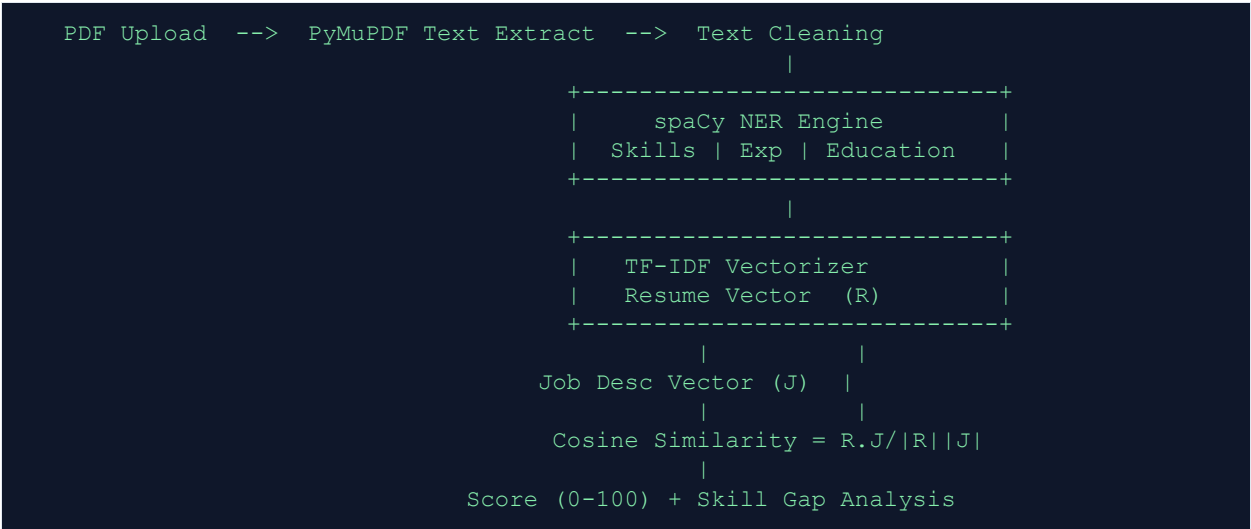
SECTION 7

AI / ML Architecture

7. AI / ML Architecture

7.1 Resume Intelligence Engine

The Resume Intelligence Engine converts raw PDF text into structured JSON. It is the data entry point for all downstream AI sub-systems.



7.2 NLP Pipeline Steps

Step 1: PDF to Text	PyMuPDF (fitz) extracts raw text from uploaded PDF. Handles multi-column layouts and removes headers/footers. Normalizes encoding to UTF-8.
Step 2: Text Cleaning	Remove URLs, phone numbers, email addresses, and special characters. Normalize whitespace. Lowercase all text for consistent matching.
Step 3: Skill Extraction	Match cleaned text against a 2,000+ curated skills dictionary using exact and fuzzy matching (rapidfuzz threshold > 85%). spaCy NER identifies organization and role entities.
Step 4: Experience Extraction	Regex patterns detect date ranges (e.g. 2020-2023) and phrases like '3 years of experience'. Sums overlapping periods. Returns total estimated years as a string.
Step 5: Education Extraction	spaCy NER ORG entities are cross-referenced with a curated education keyword list (B.Tech, M.Sc, MBA, etc.) to identify the highest qualification.

Step 6: Project Extraction	Detects section headers (Projects, Personal Projects, Portfolio) and extracts project names and associated technology stack from the following text block.
Step 7: Structured Output	Returns { skills: string[], experience: string, education: string, projects: string[] }. Stored in resumes.parsed_* columns in PostgreSQL.

7.3 Candidate Scoring System

Input	Resume JSON (parsed_skills[]) + Job Description (full text string)
Vectorization	Both inputs vectorized using a pre-fitted TfidfVectorizer trained on 10,000+ job descriptions. Output: two sparse TF-IDF vectors R and J.
Similarity	Cosine similarity: $\text{sim} = \text{dot}(R, J) / (\text{norm}(R) * \text{norm}(J))$. Returns a float in [0.0, 1.0].
Score Mapping	Final score = $\text{round}(\text{sim} * 100)$. Bands: 80-100 = Strong Match, 50-79 = Moderate Match, 0-49 = Weak Match.
Skill Gap	<code>matched_skills = intersection(resume_skills, job_required_skills)</code> . <code>missing_skills = job_required_skills minus resume_skills</code> .
Output JSON	{ score: int, matched_skills: string[], missing_skills: string[] }

7.4 Job Recommendation System

Approach	Content-Based Filtering. Matches the user skill profile vector against the pre-built active job requirements matrix.
User Vector	User skills[] array concatenated into a string and vectorized via the same TF-IDF model as scoring.
Job Matrix	All active jobs required_skills pre-vectorized and stored as a sparse matrix in Redis. Updated when new jobs are posted.
Matching	Batch <code>cosine_similarity(user_vector, job_matrix)</code> . Returns top 5 job IDs by similarity score with score included in response.
Refresh Trigger	Cache invalidated and recommendations recomputed when user updates their skills profile.
Future (v1.5)	Collaborative filtering layer: users with similar skill profiles who liked job X will also see job X recommended.

7.5 AI Interview Evaluation System

Model	Sentence-Transformers: all-MiniLM-L6-v2. Lightweight 80MB model. 384-dimensional sentence embeddings.
--------------	---

Input	{ question: string, answer: string } — the interview question and the candidate text response.
Scoring	Encode both candidate answer and the ideal answer (from curated corpus) into embedding vectors. Compute cosine similarity between them.
Keyword Bonus	Check candidate answer for presence of expected keywords from the ideal answer. Award up to +15 bonus points for keyword coverage percentage.
Score Formula	Final score = clamp((similarity * 85) + (keyword_bonus * 15), 0, 100). Always an integer.
Feedback	Rule-based: score 80+ = Excellent, 60-79 = Good but lacks depth, 40-59 = Needs more detail, < 40 = Consider revisiting fundamentals.
Confidence	high if score >= 75, medium if 50-74, low if below 50.
Output	{ score: int, feedback: string, confidence: low medium high }

SECTION 8

Security Architecture

8. Security Architecture

8.1 Authentication Flow

Token Types	Access Token (JWT, HS256, 24h expiry) + Refresh Token (opaque UUID, 30-day expiry stored in Redis)
JWT Signing	HMAC-SHA256. Secret key minimum 256-bit cryptographically random value. Rotated quarterly.
JWT Payload	{ sub: user_id, role: string, iat: int, exp: int }. Zero PII in the token payload.
Token Storage	Access token in Zustand memory (never localStorage). Refresh token in httpOnly secure cookie — not readable by JavaScript.
Auto Refresh	Axios interceptor catches 401 responses, auto-calls /auth/refresh with the cookie, retries original request with new access token.
Logout	Refresh token deleted from Redis. Client clears Zustand state. httpOnly cookie cleared by server Set-Cookie response.

8.2 Role-Based Access Control (RBAC)

Resource	Allowed Roles	Notes
Register / Login	Public	No authentication required
Browse and Search Jobs	job_seeker, recruiter, admin	Any authenticated user
Apply to a Job	job_seeker only	Recruiters blocked with HTTP 403
Create / Update / Delete Job	recruiter only	Job seekers blocked. Recruiter must own job for edit/delete.
View Resume Data	Own user (job_seeker), applicant's recruiter, admin	Cross-role access controlled by ownership checks
Update Application Status	recruiter (job owner) only	Job seekers cannot modify their own application status
All AI Endpoints	Any authenticated user	AI service trusts Core API JWT validation — no duplicate auth

Admin Panel	admin only	Separate admin role flag required in users table
--------------------	------------	--

8.3 Data Security Measures

Password Storage	bcrypt with cost factor 12. Passwords are never logged, never returned in any API response, and never stored in plain text.
Database Encryption	AWS RDS encryption at rest using AES-256. TLS 1.3 enforced on all connections to the database.
S3 Security	Bucket is private — public access block enabled. All file access via pre-signed URLs with 15-minute expiry. Server-side encryption SSE-S3 on all objects.
Environment Secrets	All credentials loaded from .env via Pydantic BaseSettings. Never hardcoded in source. .env is gitignored. Production secrets in AWS Secrets Manager.
Input Sanitization	Pydantic v2 strict validation on all inputs. SQLAlchemy ORM parameterized queries prevent SQL injection. React JSX escaping prevents XSS.
Rate Limiting	100 requests/min per IP on /auth/* endpoints. 1,000 requests/min on general API. Implemented via Redis counter with sliding window.
CORS	Strict CORS policy: only hireiq.io and localhost:3000 are allowed origins. All other origins blocked at the load balancer.
HTTPS Enforcement	HTTP connections return 301 redirect to HTTPS. HSTS header with max-age=31536000 set on all responses.

SECTION 9

Infrastructure & Deployment

9. Infrastructure & Deployment

9.1 AWS Infrastructure

VPC	Single VPC: public subnets for ALB and NAT gateway; private subnets for ECS tasks, RDS, Redis. No direct public internet to app servers.
Load Balancer	AWS Application Load Balancer. Routes /api/* to Core API ECS, /ai/* to AI ECS. Handles TLS termination and health checks.
ECS Fargate	Core API and AI Service each run as independent ECS Fargate services with auto-scaling (CPU > 70% triggers +1 task).
RDS PostgreSQL	Multi-AZ RDS PostgreSQL 15 in private subnet. Automated daily snapshots, 30-day retention. Failover in under 60 seconds.
ElastiCache Redis	Single-node Redis 7 in private subnet (v1). Cluster mode enabled in v1.5 for horizontal scaling.
S3	Private bucket with versioning enabled. Cross-region replication for disaster recovery. CloudFront for global resume file delivery.
CloudFront	Serves Next.js static assets (cache TTL 1hr) and S3 resume files globally. Origin Shield enabled to reduce origin load.
Route 53	hireiq.io → CloudFront. api.hireiq.io → ALB. Health check routing for automatic failover on zone issues.

9.2 Docker Configuration

docker-compose.yml (Local Development)

```
version: '3.9'
services:
  api:
    build: ./backend
    ports: ['8000:8000']
    env_file: ./backend/.env
    environment:
      DATABASE_URL: postgresql+asyncpg://user:pass@db:5432/hireiq
      REDIS_URL: redis://redis:6379/0
      AI_SERVICE_URL: http://ai:8001
    depends_on: [db, redis, ai]
    volumes: [./backend:/app]
```

```

ai:
  build: ./ai_service
  ports: ['8001:8001']
  volumes: [./ai_service:/app]
frontend:
  build: ./frontend
  ports: ['3000:3000']
  environment:
    NEXT_PUBLIC_API_URL: http://localhost:8000/v1
  volumes: [./frontend:/app, /app/node_modules]
db:
  image: postgres:15-alpine
  environment:
    POSTGRES_DB: hireiq
    POSTGRES_USER: user
    POSTGRES_PASSWORD: pass
  volumes: [pgdata:/var/lib/postgresql/data]
  ports: ['5432:5432']
redis:
  image: redis:7-alpine
  ports: ['6379:6379']
volumes:
  pgdata:

```

9.3 CI/CD Pipeline — GitHub Actions

Trigger	Push to main branch or pull request targeting main branch
1 — Lint	ruff check (Python) + eslint (TypeScript). Fail fast on code style violations before running tests.
2 — Test	pytest (backend, min 70% coverage) + jest (frontend). Both must pass before proceeding to build.
3 — Build	Build Docker images for all three services. Tag with git commit SHA and latest.
4 — Push	Push tagged Docker images to AWS ECR (Elastic Container Registry) in the same region as ECS.
5 — Deploy	Update ECS task definitions with new image tags. Trigger rolling deployment (zero downtime).
6 — Smoke Test	Automated smoke test suite runs against staging. Tests: auth flow, job CRUD, resume upload, AI endpoint. Rollback on failure.
7 — Notify	Deployment status (success or failure + rollback) posted to Slack #deployments channel.

SECTION 10

Environment Variables & Configuration

10. Environment Variables & Configuration

10.1 Core API — .env Reference

```
# — Application —————
APP_ENV=production          # development | staging | production
APP_HOST=0.0.0.0
APP_PORT=8000
DEBUG=false

# — Database —————
DATABASE_URL=postgresql+asyncpg://user:password@rds-host:5432/hireiq
DB_POOL_MIN=5
DB_POOL_MAX=20

# — Redis —————
REDIS_URL=redis://elasticache-host:6379/0
REDIS_TTL_DEFAULT=300

# — JWT —————
JWT_SECRET_KEY=<minimum-256-bit-cryptographically-random-secret>
JWT_ALGORITHM=HS256
JWT_ACCESS_TOKEN_EXPIRE_MINUTES=1440    # 24 hours
JWT_REFRESH_TOKEN_EXPIRE_DAYS=30

# — AWS —————
AWS_ACCESS_KEY_ID=<key>
AWS_SECRET_ACCESS_KEY=<secret>
AWS_REGION=ap-south-1
S3_BUCKET_NAME=hireiq-resumes-prod
S3_PREIGNED_URL_EXPIRY=900              # 15 minutes

# — AI Service —————
AI_SERVICE_URL=http://ai-service:8001
AI_SERVICE_TIMEOUT=10

# — Rate Limiting —————
RATE_LIMIT_AUTH=100             # requests per minute per IP on /auth/*
RATE_LIMIT_GENERAL=1000
```

10.2 Frontend — .env Reference

```
NEXT_PUBLIC_API_URL=https://api.hireiq.io/v1
NEXT_PUBLIC_APP_NAME=HireIQ
NEXT_PUBLIC_APP_ENV=production
NEXT_PUBLIC_CLOUDFRONT_URL=https://cdn.hireiq.io
NEXT_PUBLIC_MAX_RESUME_SIZE_MB=5
```

SECTION 11

Observability & Monitoring

11. Observability & Monitoring

11.1 Logging Strategy

Format	Structured JSON on all services. Fields per log line: timestamp, level, request_id, user_id, method, path, status_code, duration_ms, error (if applicable).
Library	structlog (Python) — outputs JSON in production, human-readable colored output in development.
Request ID	UUID generated per request by logging middleware. Propagated to AI service calls via X-Request-ID header for end-to-end trace correlation.
Log Levels	DEBUG (dev only) INFO (normal ops) WARNING (recoverable) ERROR (unexpected) CRITICAL (service down)
Storage	Logs shipped to AWS CloudWatch Logs. Retention: 30 days. Queryable via CloudWatch Logs Insights for incident investigation.

11.2 Metrics & Alert Thresholds

Metric	Target Threshold	Alert Condition
API Latency p95	< 500ms	Alert if > 1,000ms sustained for 5 minutes
AI Service Latency p95	< 5,000ms	Alert if > 8,000ms sustained for 3 minutes
5xx Error Rate	< 0.1%	Alert if > 1% for 2 consecutive minutes
Database CPU	< 60%	Alert if > 80% for 10 consecutive minutes
Redis Memory Usage	< 70%	Alert if > 85% — trigger cache eviction review
ECS Task Health	All tasks healthy	Alert immediately on task failure or crash loop
S3 Upload Failures	0 errors	Alert on any S3 PutObject error
Disk / Storage	< 70% full	Alert if RDS storage > 80%

11.3 Health Check Endpoints

GET /health	Core API: { status: ok, db: ok, redis: ok, uptime_s: number }. Called every 30s by ALB.
GET /ai/health	AI Service: { status: ok, model_loaded: true, latency_ms: number }. Called every 30s by ALB.
GET /api/ping	Frontend API route: returns HTTP 200. Used by CloudFront for frontend health monitoring.

HireIQ TAD v1.0 — End of Document

This Technical Architecture Document is derived from HireIQ PRD v1.0 and governs all engineering decisions.